interfaze

1.0.3 •Общедоступный• Опубликовано 7 часов назад

Бета

кода для

1 Зависимость

0 зависимостей

4 Версии

Interfaze Typescript и Javascript SDK

Официальный **Interfaze** SDK для TypeScript / JavaScript

Документы · **ограничения** · **цены** · **панель управления** · **Python SDK**

Установить

```
npm install interfaze
# или: yarn add interfaze · pnpm add interfaze · bun add inter
```

Setup

```
apiKey { ( Interfaze new =interfaze
```

```
const ; "interfaze" from }Interfaze{ import: "sk_..." }); // и
```

Ваш первый запрос

Это руководство поможет вам сделать первый запрос в Interfaze, который соответствует стандарту Chat Completions API.

```
first_name { (object . z = IdCardconst
; ) ( В интерфазе новая =интерфазе

константный ; "Зод" с }з{импорт

; "интерфазе" от }responseFormat,в интерфазе{
  импорт: з.строки(),
  фамилия: Зет.строки(),
  дата рождения: з.строки().описать("даты рождения в удостовер
  licence_number: з.строки(),
});

константные рез = ждут интерфазе.чат.пополнений.создать({
  сообщений: [
    {
      роль: "пользователь",
      содержание: [
        { тип: "текст", текст: "извлечь детали из этого id." }
        {
          тип: "url_изображения",
          url_изображения: {
            URL-адрес: "https://r2public.jigsawstack.com/inter
          },
        },
      ],
    },
  ],
  response_format: responseFormat(з.toJSONSchema(Idcardбыл), "
```

```
});

константные idcardбыл = формат JSON.синтаксический анализ(ВИЭ.
консоль.журнал(idcardбыл); // ваш Idcardбыл схемы
консоли.журнала("OCR результате:", РЭС.precontext?.[0]?.резуль
```

Precontext

Помимо ответа, ответ содержит `precontext` — необработанные метаданные, созданные Interfaze во время ответа:

```
for (const p of res.precontext ?? []) {
  console.log(p.name, p.result); // например, "ocr" -> { текст
}
```

Чат

```
сообщения { ( create .completions.chat.interfazeawait=res
  const: [
    {
      role: "user",
      content: "Какие публичные компании США отчитались о дохо
    },
  ],
});

res.choices[0]?.message.content;
```

The result is a standard `ChatCompletion` with `precontext`, and `reasoning` added (a web search backs the answer here).

Streaming

Stream the reply as it's generated; the final completion still carries `precontext` and `reasoning`.

```
const stream = interfaze.chat.completions.stream({
  messages: [
    {
      role: "user",
      content: "Summarize this week's top AI research and cite
    },
  ],
});

for await (const text of stream.textDeltas()) process.stdout.w

const final = await stream.finalChatCompletion(); // .preconte
```

`textDeltas()` yields display-ready text - the inline `<think>` / `<precontext>` side-channels are stripped and returned structured on `finalChatCompletion()`. For the raw chunk iterator, use `create({ stream: true })`.

Structured output

`responseFormat()` takes a JSON Schema - or a zod schema via `z.toJSONSchema()` - and normalizes it for Interfaze:

```
import { responseFormat, inputs } from "interfaze";
import { z } from "zod";

const Receipt = z.object({
  merchant: z.string(),
  total: z.number(),
  items: z.array(z.object({ name: z.string(), price: z.number(
});

const res = await interfaze.chat.completions.create({
  messages: [
    {
      role: "user",
      content: [{ type: "text", text: "Extract this receipt."
    },
```

```
  ],
  response_format: responseFormat(z.toJSONSchema(Receipt), "re
});

const receipt = JSON.parse(res.choices[0]?.message.content ??
```

Prefer a plain schema? Pass one directly: `responseFormat({ type: "object", properties: { … }, required: [ … ] })`. `message.content` comes back as a JSON string, so parse it - and keep the root an `object`, since a non-object root is wrapped under a `result` key.

Tools and function calling

Interfaze supports tools - define `tools`, read `message.tool_calls`, run them, then pass the results back for the final answer.

```
import type { ChatCompletionMessageParam, ChatCompletionTool }

const tools: ChatCompletionTool[] = [
  {
    type: "function",
    function: {
      name: "get_weather",
      description: "Get the current weather for a city.",
      parameters: {
        type: "object",
        properties: { city: { type: "string", description: "e.
        required: ["city"],
      },
    },
  },
];

const messages: ChatCompletionMessageParam[] = [{ role: "user"

const res = await interfaze.chat.completions.create({
  messages,
```

```
  tools,
  tool_choice: "auto",
});

const message = res.choices[0]?.message;
if (message) messages.push(message); // the assistant turn, ca

for (const call of message?.tool_calls ?? []) {
  if (call.type !== "function") continue;
  const { city } = JSON.parse(call.function.arguments);
  messages.push({
    role: "tool",
    tool_call_id: call.id,
    content: await getWeather(city),
  });
}

// send the tool results back for the final answer
const final = await interfaze.chat.completions.create({
  messages,
  tools,
  tool_choice: "auto",
});
```

Reasoning

Ask for reasoning with `reasoning_effort`; the text comes back on `res.reasoning`.

```
const res = await interfaze.chat.completions.create({
  reasoning_effort: "high", // also accepts Interfaze's "on" /
  messages: [
    {
      role: "user",
      content: "Which region should we launch in first, and wh
    },
```

```
  ],
});

res.reasoning; // reasoning text - present with reasoning_effo
```

Multimodal Inputs

Interfaze handles images, PDFs, audio, video, and CSV. The simplest way is to drop a public URL into the prompt - Interfaze fetches and reads it:

```
await interfaze.chat.completions.create({
  messages: [
    {
      role: "user",
      content: "Summarize this document: https://arxiv.org/pdf
    },
  ],
});
```

Or attach it as a content part - a `file` part for documents, audio, and video; `image_url` for images:

```
await interfaze.chat.completions.create({
  messages: [
    {
      role: "user",
      content: [
        { type: "text", text: "Summarize this document." },
        {
          type: "file",
          file: {
            filename: "paper.pdf",
            file_data: "https://arxiv.org/pdf/1706.03762",
          },
        },
```

```
        ],
      },
    ],
});
```

`file_data` also takes a base64 data URI:

```
{ type: "file", file: { filename: "report.pdf", file_data: `da
```

`inputs.*` is a typed shortcut that builds these parts - and turns raw bytes or a local file into a data URI for you:

```
import { inputs } from "interfaze";

inputs.image("https://…/photo.png"); // image_url part
inputs.file("https://…/report.pdf"); // file part
inputs.audio("https://…/call.wav"); // input_audio part
inputs.file(await inputs.dataUrl(pdfBytes, "application/pdf"),
  filename: "report.pdf",
}); // bytes / Blob
inputs.image(await inputs.fromPath("./photo.png")); // Node: r
```

Interfaze rejects `image/gif` and `image/avif` client-side, and `inputs.fromPath` is Node-only.

Tasks

A task (**run_task**) runs one built-in tool instead of the full model - faster and cheaper, but limited to that tool and its fixed output structure. So `task` can't be combined with a custom `response_format`; reach for a full completion (like **your first request**) when you need the whole model or your own schema.

The `tasks.*` helpers are the shortest way - each takes a source and returns the raw result (typed `unknown`, so validate before use):

```
await interfaze.tasks.ocr(url);
await interfaze.tasks.objectDetection(url);
await interfaze.tasks.guiDetection(url);
await interfaze.tasks.webSearch(query);
await interfaze.tasks.scrape(url);
await interfaze.tasks.transcribe(url);
await interfaze.tasks.translate(text, { to: "Spanish" });
await interfaze.tasks.forecast(csvUrl, { periods: 30, unit: "d
```

For a multi-part message (text plus an input), set `task` on a normal `create` call - the result comes back on `message.content`:

```
const res = await interfaze.chat.completions.create({
  task: "ocr",
  messages: [
    {
      role: "user",
      content: [{ type: "text", text: "Extract the total." },
    },
  ],
});
const { result } = JSON.parse(res.choices[0]?.message.content
```

Or a `<task>…</task>` system message plus an empty response schema:

```
import { emptyTaskSchema } from "interfaze";

const res = await interfaze.chat.completions.create({
  messages: [
    { role: "system", content: "<task>ocr</task>" },
    {
      role: "user",
      content: [{ type: "text", text: "Extract the total." },
    },
  ],
```

```
  response_format: emptyTaskSchema(), // an empty JSON schema
});
const { result } = JSON.parse(res.choices[0]?.message.content
```

Guardrails

Enable safety categories with `guard`; a blocked request comes back as a normal completion, not an exception.

```
const res = await interfaze.chat.completions.create({
  guard: ["S1", "S10", "S12_IMAGE"],
  messages: [{ role: "user", content: "..." }],
});
```

A match returns the plain string `unsafe S1` as `message.content` - so check for it. See the exported `GUARD_CODES` / `GUARD_LABELS`.

Client options

Set router, cache, and streaming behavior once on the client:

```
const interfaze = new Interfaze({
  showAdditionalInfo: true, // stream <precontext> deltas as t
  bypassMoA: true, // skip the mixture-of-architecture router
  bypassCache: true, // skip the semantic cache
});
```

> **Zero data retention (ZDR)** is configured at the account/infrastructure level, not via a per-request flag or client header - contact Interfaze to enable it for your account.

Errors

Interfaze re-exports typed error classes to catch and narrow on:

```
import { BadRequestError, InterfazeError, RateLimitError } frc
```

`InterfazeError` is client-side (missing key, invalid guard code, stream misuse). Everything else is an `APIError` subclass carrying `status` and `code` - `BadRequestError` (400), `AuthenticationError` (401), `RateLimitError` (429), and so on.

Capabilities

Use case	Entry point
Chat	`chat.completions.create`
Streaming	`chat.completions.stream`
Structured output	`responseFormat()`
Reasoning	`reasoning_effort`
Tools	`tools`
Multimodal inputs	`inputs.*`
OCR	`tasks.ocr`
Object and GUI detection	`tasks.objectDetection`, `tasks.guiDetection`
Web search and scraping	`tasks.webSearch`, `tasks.scrape`
Speech to text	`tasks.transcribe`
Translation	`tasks.translate`
Forecasting	`tasks.forecast`
Guardrails	`guard`
Precontext	`res.precontext`

Examples

Runnable snippets in `examples/`.

License

MIT

Keywords

interfaze openai ai ocr speech-to-text web-search sdk

Provenance

Built and signed on
GitHub Actions
View build summary

Source Commit
github.com/InterfazeAI/interfaze-js@8024031

Build File
.github/workflows/publish.yml

Public Ledger
Transparency log entry

Share feedback

Install

```
> npm i interfaze
```

Repository

github.com/InterfazeAI/interfaze-js

Homepage

interfaze.ai

Weekly Downloads

171

Version

1.0.3

License

MIT

Issues

0

Pull Requests

1

Last publish

7 hours ago

Collaborators

Analyze security with Socket

Check bundle size

View package health

Explore dependencies

Report malware

Support

Help

Advisories

Status

Contact npm

Company

About

Blog

Press

Terms & Policies

Policies

Terms of Use

Code of Conduct

Privacy